

Phase 3 Build Report — graphdb_rs

Date: 2026-08-24
Location: /home/ubuntu/graphdb_rs/
Status:  All steps completed successfully

Phase 3 delivers a production-quality Rust port of the Phase-1 graph DB core, plus a PyO3 Python FFI bridge that the Phase-2 ai_memory layer can optionally use. (Phases 1 & 2 remain intact at /home/ubuntu/graphdb .)

Toolchain

Tool	Version
rustc	1.98.0
cargo	1.98.0
Python	3.11.6
maturin	1.15.0

Rust was not pre-installed; it was installed via rustup during this phase.

Step 1 — Rust installed

rustc --version → rustc 1.98.0 . Installed via the official rustup script.

Step 2 — cargo build

```
Compiling graphdb_rs v0.1.0 (/home/ubuntu/graphdb_rs)
  Finished `dev` profile [unoptimized + debuginfo] target(s)
```

Compiles cleanly with **no warnings**. cargo clippy --all-targets also passes with no warnings.

Step 3 — cargo test

```
running 11 tests
test test_cosine_similarity_correctness ... ok
test test_delete_node_cascades_edges ... ok
test test_label_index_and_nodes_by_label ... ok
test test_bfs_dfs_order ... ok
test test_find_path ... ok
test test_top_k_similar_ordering ... ok
test test_similarity_is_label_scoped ... ok
test test_similarity_scanner_finds_match_above_threshold ... ok
test test_similarity_respects_edge_label ... ok
test test_search_similar_nodes_label_scoped ... ok
test test_save_load_round_trip ... ok

test result: ok. 11 passed; 0 failed
```

Tests cover: label indexing, similarity scanner matches above threshold, label-scoped isolation (different labels are never compared), edge-label filtering, BFS/DFS ordering + depth limits, shortest-path (incl. unreachable and self-path), save/load round-trip, top-k ordering, cosine correctness (parallel = 1.0, orthogonal = 0.0, opposite = -1.0, zero-vector = 0.0, dimension mismatch = error), and cascade edge deletion.

Step 4 — Python extension build

`maturin develop` requires an active virtualenv (none present here), so a wheel was built and installed instead:

```
maturin build --release
📦 Built wheel for CPython 3.11 to
  target/wheels/graphdb_rs-0.1.0-cp311-cp311-manylinux_2_34_x86_64.whl
pip install --force-reinstall target/wheels/graphdb_rs-0.1.0-*.whl
Successfully installed graphdb-rs-0.1.0
```

`build.sh` uses `maturin develop --release` (the venv path); the wheel route above is the no-venv fallback documented in `README_RUST.md`.

Step 5 — Python smoke test

`python3 smoke_test.py` :

```
graphdb_rs version: 0.1.0
node_count: 4
was_flagged: True similar: 1
search_similar_nodes(User): [(<alice_id>, 1.0), (<bob_id>, 0.99994...)]

ALL SMOKE TESTS PASSED ✓
```

Verified through the compiled Rust extension: node/edge CRUD, typed property round-trip (str/int/float), embeddings, label-scoped similarity flagging on `add_edge`, label-scoped vector search, BFS/DFS/find_path, `KeyError` on missing node (error mapping), and JSON persistence round-trip.

Bridge backend selection

```
bridge backend: rust          # when the compiled .so is importable
fallback backend: python      # when graphdb_rs import is blocked
fallback store type: graphdb.store
```

`python_bridge/bridge.py` correctly prefers the Rust backend and transparently falls back to the pure-Python `graphdb` package when the extension is absent.

Critical constraints — verification

1. **✓ Compiles cleanly** with `cargo build` (and `cargo clippy`), Rust installed via `rustup`.
2. **✓ Label-scoped similarity mirrors Python** — only nodes with the same text label are compared (`test_similarity_is_label_scoped`).
3. **✓ `parking_lot::RwLock`** used for the store (not `std::sync::RwLock`).
4. **✓ All public Rust APIs return `Result<T, GraphError>`** — no panics in library code.
5. **✓ PyO3 maps errors** — `NodeNotFound` / `EdgeNotFound` → `KeyError`, `DimensionMismatch` → `ValueError`, `Io` → `IOError`, `Serde` → `ValueError`.
6. **✓ Python fallback bridge works** without the Rust `.so` (verified above).

Deliverables

```
graphdb_rs/
  Cargo.toml          pyproject.toml    build.sh
  src/ lib.rs core.rs store.rs vector.rs similarity.rs index.rs persistence.rs
  query.rs error.rs
  tests/ integration_test.rs             (11 tests, all passing)
  python_bridge/ __init__.py bridge.py graphdb_rs.pyi
  README_RUST.md      BUILD_REPORT.md   smoke_test.py
```

How to reproduce

```
source $HOME/.cargo/env
cd /home/ubuntu/graphdb_rs
cargo build && cargo test          # Rust core + 11 integration tests
./build.sh                        # build + install the Python extension (venv)
# or, without a venv:
# maturin build --release && pip install --force-reinstall target/wheels/*.whl
python3 smoke_test.py             # Python smoke test through the Rust engine
```